3-2 Milestone Two: Enhancement One: Software Design and Engineering

Andrew Allen

Department of Computer Science, Southern New Hampshire University

CS 499: Computer Science Capstone

Dr. Sherin Aly

July 19th, 2026

Enhancement One: Software Design and Engineering

Chosen Artifact

The artifact I chose for my ePortfolio is an Android inventory tracking app that I originally developed during my time in CS360 Mobile Architect and Programming course. The application allowed users to manage inventory items by creating, viewing, updating, and deleting products while tracking quantities and product information. It was developed using Java, Android Studio, and the Model-View-Controller (MVC) design pattern.

For this enhancement, I redesigned and rebuilt the whole application from the ground up using Kotlin, the Model-View-ViewModel (MVVM) design pattern, and Jetpack navigation components. This approach separates the user interface from the applications business logic, creating a more modular and maintainable codebase easier to expand and test.

Justification for inclusion

I chose this artifact because it demonstrates my ability to apply software engineering principles to redesign and rebuild an application using modern Android standards. While the original application served as the foundation, I recreated the application from scratch, migrating the codebase from Java to Kotlin, redesigning the navigation, replacing the original activity-based structure with a fragment-based architecture, and implementing the MVVM design pattern to create a more maintainable and modular architecture. These enhancements allowed me to demonstrate my programming, critical thinking, and problem-solving skills to create a more effective solution.

Course Outcome Coverage

The enhancements completed for this artifact successfully meet the software engineering and design outcomes I established for the artifact. I successfully ported the application to another language, implemented a better design architecture, and made the codebase more modular and maintainable.

As for updates to my outcome/coverage plans, there are a couple features from the original application that I plan to incorporate in future versions. The original artifact utilizes a login screen and SMS notification for low stock alerts. I am planning to keep the login screen as a secondary way for users to be able to save their inventories to their Google Drive accounts. I have also decided I would prefer to implement standard android notifications as opposed to SMS notifications, as it is better practice. Also, I have not implemented barcode scanning yet to populate the SKU field but plan to in a later version. I would also like to implement the camera for taking photos of inventory items and displaying them in the inventory list.

Reflection

Enhancing this artifact reinforced the importance of designing software with maintainability and scalability in mind. Rather than focusing only on adding features, I focused on learning how to improve the underlying architecture to make the application more reliable and maintainable for future modifications. Throughout this enhancement, I also developed a deeper understanding of the MVVM design pattern and how it separates responsibilities within an application. I managed the applications state through a shared ViewModel which allowed different parts of the app to communicate independently from the user interface.

The main challenges I faced during development were navigation, lifecycle, and dependency issues. Before I could get the app to build, I had to double check and make sure I

had all the proper imports. Thankfully, Android Studio is very smart and helps with this by highlighting errors in red and providing a means to quickly import. Then, when I got the application to build, I began testing it in the emulator. The first thing I noticed after launching the application and trying to return to the InventoryFragment from the ItemDetailFragment was that it would take me to the home screen. To solve this problem, I had to configure the OnBackPressedDispatcher in MainActivity to work with the applications NavController. This ensured Androids navigation system remained in sync with Jetpacks navigation.

Another challenge I faced while working on this enhancement was scope creep. As I was porting over the codebase to Kotlin, I would naturally think of ideas and features that I would like to implement into the application. Rather than continually adding new features, I prioritized building a stable architectural foundation first. This experience reinforced the importance of controlling project scope and completing core functionality before expanding the applications features.